

09_tarea_heatmap_hexabin_diamonds

April 27, 2026

1 Tarea paso a paso: Heatmap 2D y Hexbin

¡ATENCIÓN! Lee esto antes de empezar: Esta tarea está diseñada paso a paso para que afiances los conceptos vistos en clase sobre los gráficos de densidad **Heatmap 2D** y **Hexbin**. Su objetivo es que practiques cómo visualizar la **densidad de datos** cuando tienes tantos puntos que un scatter plot clásico se convierte en una mancha ilegible.

Nota importante sobre la Inteligencia Artificial: Es tentador copiar y pegar el enunciado de estos ejercicios en ChatGPT o Claude para que te den la solución... **Hacer eso perjudicará gravemente tu aprendizaje. (CODEA UN POCO QUE TE ENTERES QUE HACES).** La única forma de asimilar la sintaxis de las herramientas de visualización es escribiéndolas. **Usa tu cerebro, actúa paso a paso, equivócate, lee la documentación de Seaborn y Pandas, busca tu fallo y ¡aprende de él!**

1.1 Preparación y Datos

Para esta tarea vamos a utilizar el dataset **“Diamonds”**, que recoge las características y precios de más de **53.000 diamantes**. Cada fila representa un diamante con su peso en quilates, su corte, color, claridad y precio en dólares.

Con más de 53.000 puntos, es el dataset ideal para ver el problema del **overplotting**: si intentamos hacer un scatter plot clásico de **carat vs price**, los puntos se solaparán tanto que será imposible saber dónde se concentran realmente los datos. Los gráficos de densidad (Heatmap 2D y Hexbin) son la solución.

1. Descarga el dataset de Kaggle: [Diamonds](#)
2. Guarda el archivo `diamonds.csv` en la carpeta `data/` de tu proyecto.
3. En la siguiente celda, importa las librerías necesarias y carga el CSV en un DataFrame llamado `df`.

```
[1]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# CARGA DE DATOS
df = pd.read_csv("data/diamonds.csv")

df.head(5)
```

```
[1]:   carat      cut color clarity depth table price      x      y      z
0    0.23    Ideal      E     SI2   61.5   55.0   326   3.95   3.98   2.43
1    0.21  Premium      E     SI1   59.8   61.0   326   3.89   3.84   2.31
2    0.23     Good      E     VS1   56.9   65.0   327   4.05   4.07   2.31
3    0.29  Premium      I     VS2   62.4   58.0   334   4.20   4.23   2.63
4    0.31     Good      J     SI2   63.3   58.0   335   4.34   4.35   2.75
```

```
[2]: # Ver la información del dataset
df.describe(include="all")
```

```
[2]:   count      carat      cut color clarity      depth      table \
count    53940.000000    53940    53940    53940    53940.000000    53940.000000
unique           NaN         5         7         8           NaN           NaN
top           NaN    Ideal         G     SI1           NaN           NaN
freq           NaN    21551    11292    13065           NaN           NaN
mean         0.797940         NaN         NaN         NaN        61.749405        57.457184
std         0.474011         NaN         NaN         NaN         1.432621         2.234491
min         0.200000         NaN         NaN         NaN        43.000000        43.000000
25%         0.400000         NaN         NaN         NaN        61.000000        56.000000
50%         0.700000         NaN         NaN         NaN        61.800000        57.000000
75%         1.040000         NaN         NaN         NaN        62.500000        59.000000
max         5.010000         NaN         NaN         NaN        79.000000        95.000000

count      price      x      y      z
count    53940.000000    53940.000000    53940.000000    53940.000000
unique           NaN           NaN           NaN           NaN
top           NaN           NaN           NaN           NaN
freq           NaN           NaN           NaN           NaN
mean     3932.799722         5.731157         5.734526         3.538734
std     3989.439738         1.121761         1.142135         0.705699
min       326.000000         0.000000         0.000000         0.000000
25%      950.000000         4.710000         4.720000         2.910000
50%     2401.000000         5.700000         5.710000         3.530000
75%     5324.250000         6.540000         6.540000         4.040000
max    18823.000000        10.740000        58.900000        31.800000
```

```
[3]: # Limpieza de nulos
df = df.dropna()
df
```

```
[3]:   carat      cut color clarity depth table price      x      y      z
0    0.23    Ideal      E     SI2   61.5   55.0   326   3.95   3.98   2.43
1    0.21  Premium      E     SI1   59.8   61.0   326   3.89   3.84   2.31
2    0.23     Good      E     VS1   56.9   65.0   327   4.05   4.07   2.31
3    0.29  Premium      I     VS2   62.4   58.0   334   4.20   4.23   2.63
4    0.31     Good      J     SI2   63.3   58.0   335   4.34   4.35   2.75
...    ...    ...    ...    ...    ...    ...    ...    ...    ...
```

53935	0.72	Ideal	D	SI1	60.8	57.0	2757	5.75	5.76	3.50
53936	0.72	Good	D	SI1	63.1	55.0	2757	5.69	5.75	3.61
53937	0.70	Very Good	D	SI1	62.8	60.0	2757	5.66	5.68	3.56
53938	0.86	Premium	H	SI2	61.0	58.0	2757	6.15	6.12	3.74
53939	0.75	Ideal	D	SI2	62.2	55.0	2757	5.83	5.87	3.64

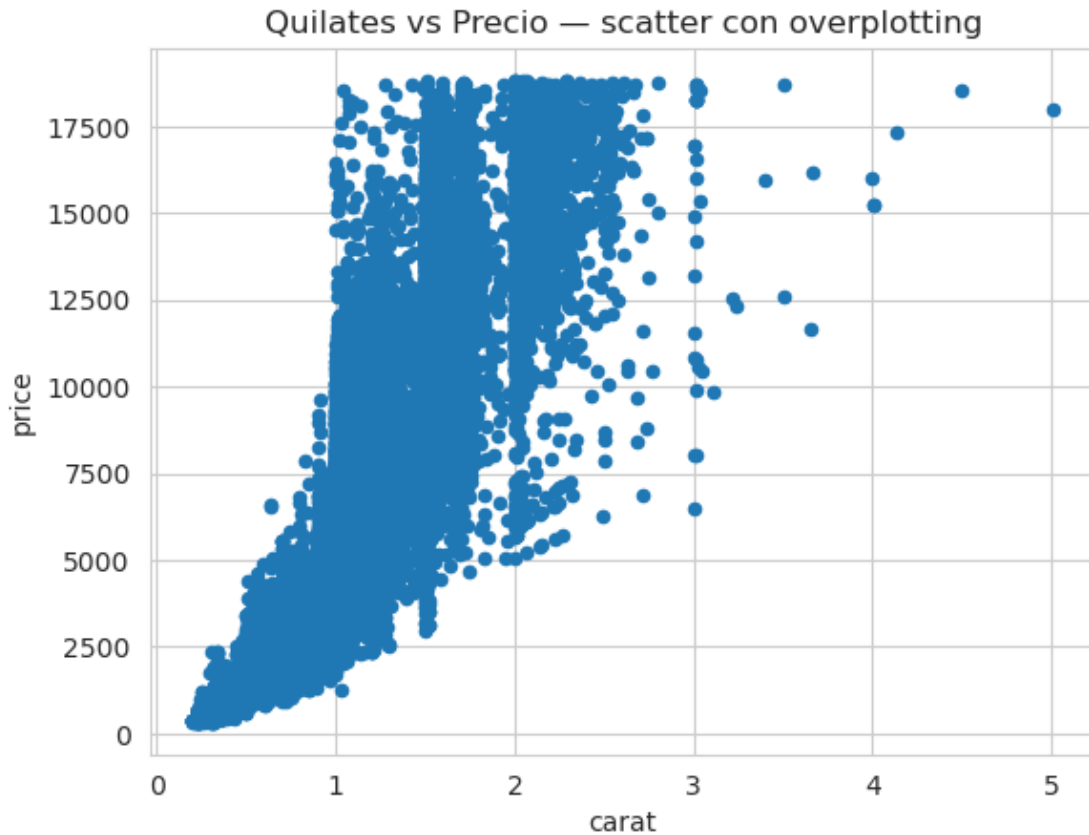
[53940 rows x 10 columns]

1.2 Paso 1: El Problema — Scatter Plot con Overplotting

Objetivo: Ver con tus propios ojos por qué el scatter plot clásico falla con datasets grandes y necesitamos una alternativa de densidad.

Instrucciones: 1. Usa `df.plot.scatter()` para hacer un scatter plot con `x='carat'` e `y='price'`. 2. Añade un título con `plt.title(...)` y muestra el gráfico. 3. Observa el resultado: ¿puedes saber dónde se concentran la mayoría de los diamantes? ¿O todo es una mancha azul oscura? Ese es el problema del **overplotting**.

```
[4]: # carat vs price con todos los puntos (53k) - para ver el problema del
      ↪overplotting
      # https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.plot.scatter.
      ↪html
      sns.set_style("whitegrid")
      df.plot.scatter(
          x='carat',
          y='price'
      )
      plt.title('Quilates vs Precio - scatter con overplotting')
      plt.show()
```



1.2.1 Interpretación

Con 53.000 puntos superpuestos, el scatter plot es casi inútil: no se distingue dónde están la mayoría de los diamantes. Necesitamos un gráfico que muestre la **densidad** de puntos en cada zona, no los puntos individuales.

1.3 Paso 2: Heatmap 2D con Jointplot (kind='hist')

Objetivo: Usar el `jointplot` de Seaborn con `kind='hist'` para dividir el espacio en celdas rectangulares y colorear cada celda según la densidad de puntos que contiene.

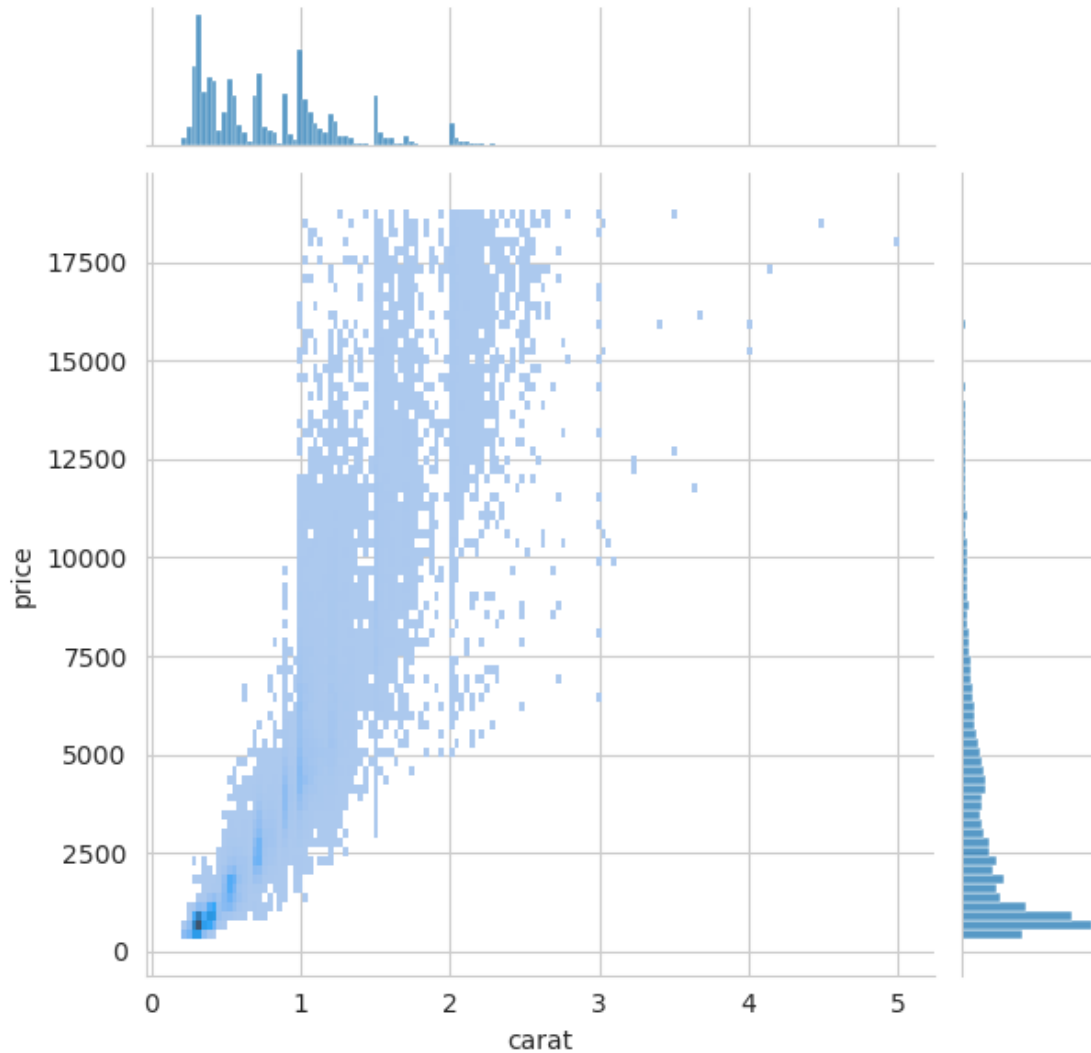
Instrucciones: 1. Usa `sns.jointplot()` con `data=df`, `x='carat'`, `y='price'` y `kind='hist'`. 2. Añade `plt.show()` al final. 3. Observa cómo las celdas más oscuras indican las zonas donde se concentran más diamantes. Los gráficos laterales muestran la distribución marginal de cada variable por separado.

```
[5]: # Heatmap de datos numéricos (rectángulos + histogramas marginales)
# https://seaborn.pydata.org/generated/seaborn.jointplot.html
sns.jointplot(
    data=df,
    x='carat',
```

```

    y='price',
    kind='hist'
)
plt.show()

```

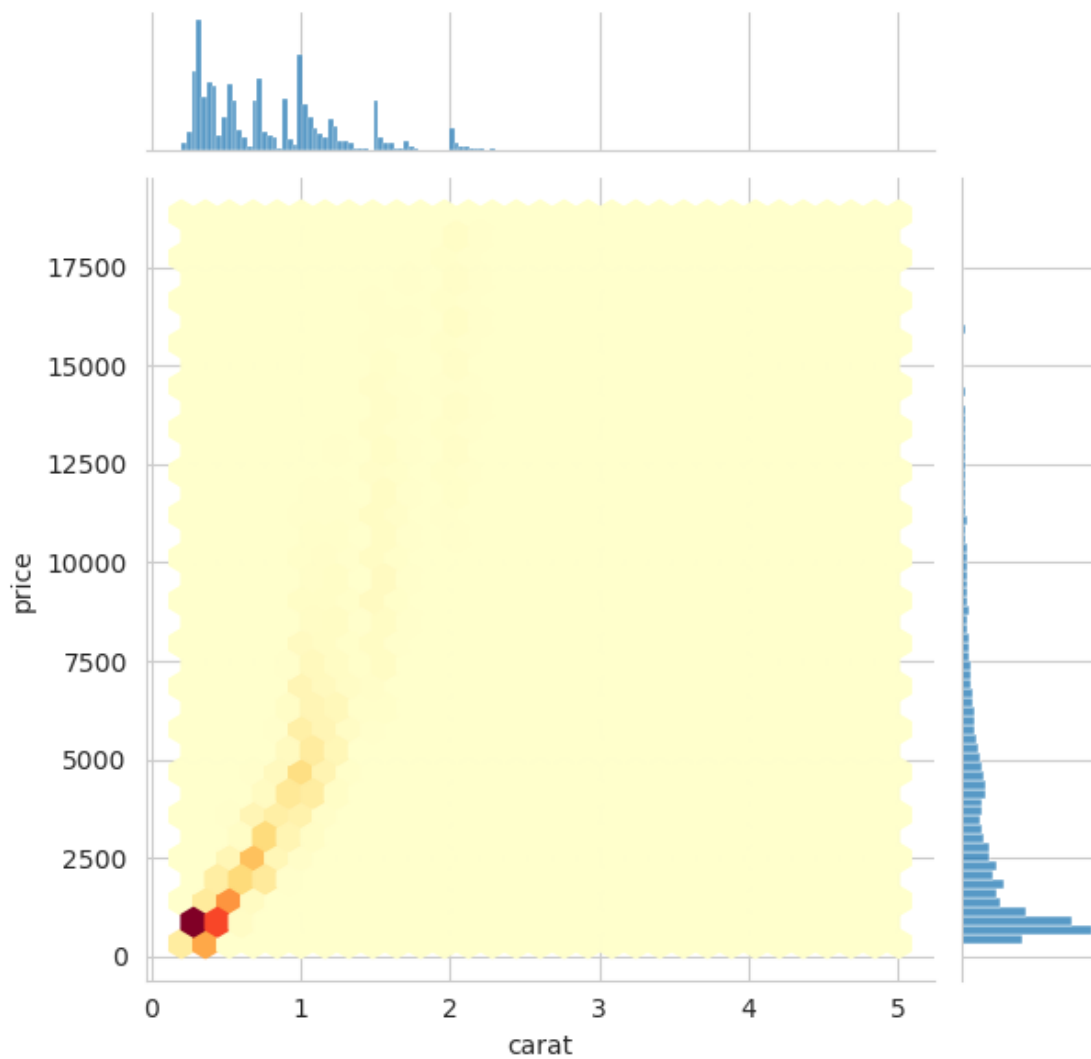


1.4 Paso 3: Hexbin con Jointplot (kind='hex')

Objetivo: Sustituir las celdas rectangulares por hexágonos, que tesslan el plano de forma más eficiente y son visualmente más claros para representar densidad.

Instrucciones: 1. Repite el `sns.jointplot()` del Paso 2 pero cambia `kind='hex'`. 2. Dentro del parámetro `joint_kws` (un diccionario), añade: - `'gridsize': 30` para controlar el tamaño de los hexágonos (más alto = hexágonos más pequeños y más detalle). - `'cmap': 'YlOrRd'` para usar una paleta de colores de amarillo a rojo (amarillo = poca densidad, rojo = mucha densidad). 3. Muestra el gráfico. ¿Qué rango de quilates y precios concentra más diamantes?

```
[6]: # Heatmap de datos numéricos en versión hexagonal
# https://seaborn.pydata.org/generated/seaborn.jointplot.html
sns.jointplot(
    data=df,
    x='carat',
    y='price',
    kind='hex',
    joint_kws={
        'gridsize': 30,          # más alto = hexágonos más pequeños y más
    detail
        'cmap': 'YlOrRd'        # paleta amarillo-rojo para densidad
    }
)
plt.show()
```

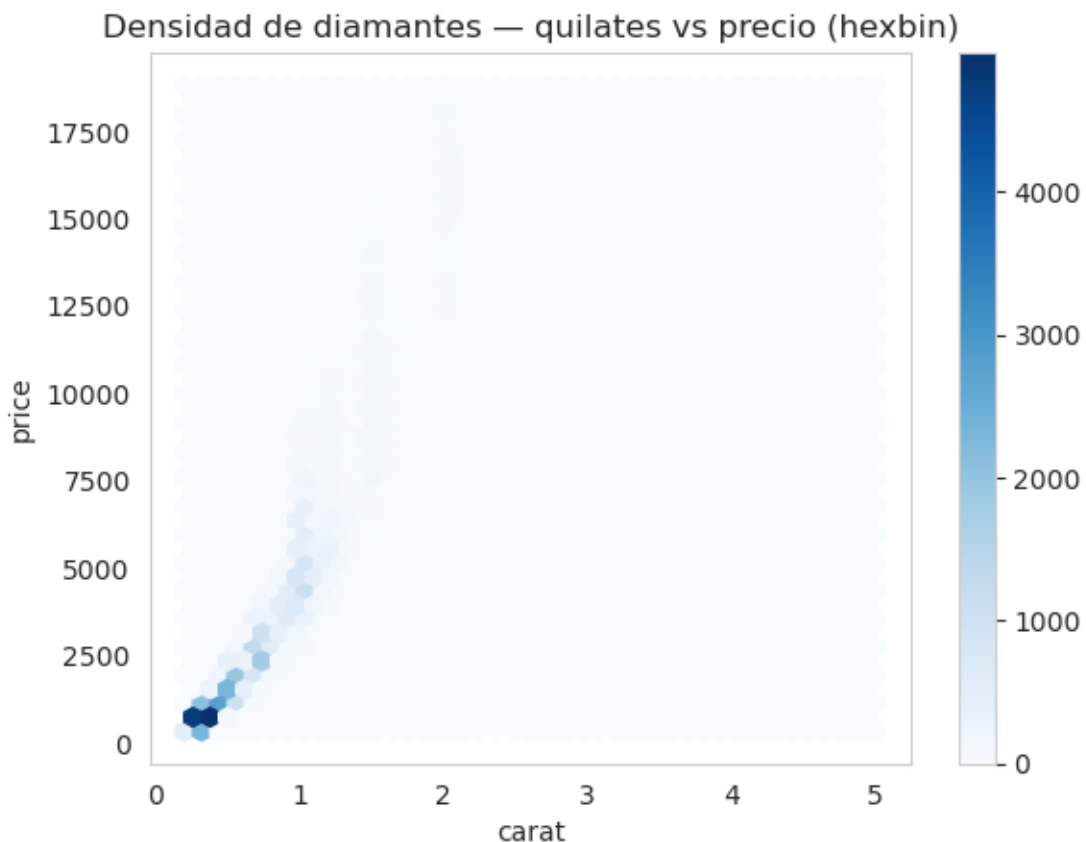


1.5 Paso 4: Hexbin con Pandas y Personalización

Objetivo: Crear el hexbin directamente desde Pandas con `df.plot.hexbin()`, que es la forma más rápida para exploración inicial, y personalizar el mapa de colores y el tamaño de la cuadrícula.

Instrucciones: 1. Usa `df.plot.hexbin()` con `x='carat'`, `y='price'` y `gridsize=40`. 2. Añade el parámetro `cmap='Blues'` para usar una paleta de azules (o prueba `'inferno'`, `'plasma'`, `'magma'`). 3. Desactiva la cuadrícula de fondo con `plt.grid(False)` para que no interfiera visualmente con los hexágonos. 4. Añade un título descriptivo y muestra el gráfico. 5. (Opcional): prueba a cambiar `gridsize` a 15 y a 60 y observa cómo afecta al nivel de detalle.

```
[7]: # Hexabin de datos numéricos
# https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.plot.hexbin.html
df.plot.hexbin(
    x='carat',
    y='price',
    gridsize=40,                                # algo más fino que el paso anterior
    cmap='Blues'
)
plt.grid(False)                                # sin rejilla, que compite con los hexágonos
plt.title('Densidad de diamantes - quilates vs precio (hexbin)')
plt.show()
```



1.5.1 Interpretación del Hexbin

Los gráficos de densidad revelan lo que el scatter plot ocultaba:

1. **La gran mayoría de los diamantes** se concentra en la zona de **0.3 a 1.0 quilates** y **500 a 5.000 dólares**. Esa es la zona de mayor demanda del mercado.
2. **A mayor número de quilates, mayor precio**, pero la relación no es lineal: los diamantes de más de 2 quilates son muy escasos (hexágonos aislados en la esquina superior derecha).
3. El **parámetro gridsize** controla la resolución: un valor bajo da una visión general (hexágonos grandes), un valor alto da más detalle pero puede ser ruidoso si los datos son escasos en alguna zona.

1.6 Reflexión Final del Alumno

Tómate un rato para reflexionar. * El **Heatmap 2D** y el **Hexbin** son los gráficos de referencia cuando trabajas con datasets con **más de 10.000 filas** y quieres estudiar la relación entre dos variables continuas. El scatter plot clásico colapsa por overplotting, pero estos gráficos revelan la estructura real de los datos. * La diferencia entre ambos es geométrica: el histograma 2D usa **rectángulos** (más fácil de leer en cuadrículas alineadas) y el hexbin usa **hexágonos** (tesslan el plano sin dejar huecos y tienen distancia uniforme al centro en todas las direcciones). * Como reto adicional: ¿podrías repetir el hexbin pero filtrando solo los diamantes con `cut == 'Ideal'`? ¿La distribución de quilates y precios cambia respecto al dataset completo?

Asegúrate de que cada celda tenga sus respectivos comentarios.

La forma de la nube no cambiaría mucho, pero se iría un poco arriba en Y. A igualdad de quilates un Ideal se paga más caro que un corte inferior.

```
[8]: # Reto: hexbin solo con cut == 'Ideal' (espero la nube algo más arriba en Y, ↵
      ↪son más caros)
df_ideal = df[df['cut'] == 'Ideal']
df_ideal.plot.hexbin(
    x='carat',
    y='price',
    gridsize=40,
    cmap='Blues'
)
plt.grid(False)
plt.title("Densidad de diamantes de corte Ideal - quilates vs precio")
plt.show()
```